FITBUDDY – AI FITNESS PLAN GENERATOR USING GEMINI MODELS

Project Work Assignment

Team ID	unm293200909
Project Name	FitBuddy-AI fitness plan generator using Gemini models
Team Member	Kavyasri.A
Assigned Work	All project activities assigned in the given milestone/order

Project Overview

FitBuddy is a web-based application that uses AI to generate personalized workout plans and nutrition tips based on fitness goals such as weight loss, muscle gain, or general wellness. It uses FastAPI with Google Gemini AI models, SQLite with SQLAlchemy for storage, and Jinja2 templates for the web interface. The system generates a customized 7-day fitness plan and can refine the plan using user feedback.

Work Assignment – According to Milestone Order

S.No	Milestone / Activity	Work Assigned to Kavyasri.A
1	**1** Milestone 1 – Model Selection and Architecture	Handle model selection and application architecture planning.
2	**2** Activity 1.1 – Research and Select the Appropriate Generative AI Model	Research project requirements, evaluate suitable Gemini models, and select Gemini 1.5 Pro for workout generation/updates and Gemini Flash for nutrition tips.
3	**3** Activity 1.2 – Define the Architecture of the Application	Define the modular architecture covering HTML/Jinja2 frontend, FastAPI backend, Gemini AI layer, and SQLite/SQLAlchemy database.
4	**4** Activity 1.3 – Set Up the Development Environment	Set up Python, virtual environment, required libraries, project folders, Uvicorn server, and local application environment.
5	**5** Milestone 2 – Core Functionalities Development	Coordinate development of the main FitBuddy functionality.
6	**6** Activity 2.1 – Develop the Core Functionalities	Implement workout plan generation, nutrition/recovery tip generation, feedback-based plan updating, and user/plan storage.
7	**7** Activity 2.2 – Implement FastAPI Backend	Implement routes, process form inputs, validate data with Pydantic schemas, connect Gemini functions, and return generated results to templates.
8	**8** Milestone 3 – routes.py Development	Develop the main routing and operational logic connecting frontend, Gemini modules, and database.
9	**9** Activity 3.1 – Writing the Main Application Logic in routes.py	Implement /, /generate-workout, /submit-feedback, and /view-all-users routes; connect AI functions and database operations.
10	**10** Milestone 4 – Frontend Development	Develop the user-friendly and responsive FitBuddy web interface.
11	**11** Activity 4.1 – Designing and Developing User Interface	Create index.html, result.html, and all_users.html; design responsive layouts, forms, buttons, result blocks, and fitness-oriented styling.
12	**12** Activity 4.2 – Creating Dynamic Templates with FastAPI's Jinja2	Configure Jinja2Templates, bind backend data to HTML templates, display workout/nutrition results, and implement feedback submission.
13	**13** Milestone 5 – Deployment	Prepare and verify FitBuddy for local deployment.
14	**14** Activity 5.1 – Preparing the Application for Local Deployment	Set up virtual environment, install dependencies, configure Gemini API key through environment variables, and verify template rendering.

S.No	Milestone / Activity	Work Assigned to Kavyasri.A
15	**15** Activity 5.2 – Testing and Verifying Local Deployment	Run the FastAPI/Uvicorn server, test the application through the browser and /docs endpoint, and verify the main user flows.

Key Features Covered

• Personalized 7-day workout plan based on user details, goal, and intensity.

• Nutrition or recovery tip generated using Gemini Flash.

• Feedback-based workout plan refinement using Gemini 1.5 Pro.

• SQLite/SQLAlchemy storage of user information and original/updated plans.

• Admin view for registered users and their workout plans.

• FastAPI backend with Jinja2-powered HTML interface and local deployment/testing.

Source Basis

This assignment follows the milestone and activity order documented in the uploaded FitBuddy project document. The project document defines five milestones covering model selection and architecture, core functionality, routes.py development, frontend development, and deployment. It also describes the Gemini Pro/Flash integration, FastAPI backend, SQLite database, Jinja2 templates, and local testing workflow.